HOG Descriptors

HOG, or Histogram of Oriented Gradients, is a feature descriptor that is often used to extract features from image data.

- The HOG descriptor focuses on the structure or the shape of an object. In the case of edge features, we only identify if the pixel is an edge or not. HOG is able to provide the edge direction as well. This is done by extracting the **gradient and orientation** (magnitude and direction) of the edges
- These orientations are calculated in **'localized' portions**. This means that the complete image is broken down into smaller regions and for each region, the gradients and orientation are calculated.
- Finally the HOG would generate a **Histogram** for each of these regions separately. The histograms are created using the gradients and orientations of the pixel values, hence the name 'Histogram of Oriented Gradients'

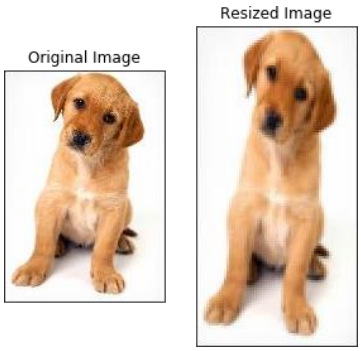
To put a formal definition to this:

> *The HOG feature descriptor counts the occurrences of gradient orientation in localized portions of an image.*

Process of Calculating the Histogram of Oriented Gradients (HOG)

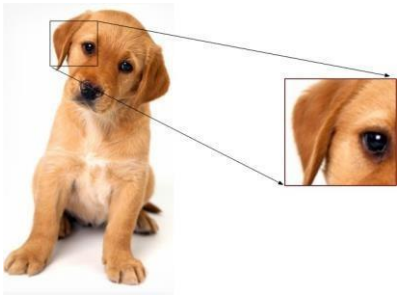
Step 1: Preprocess the Data (64 x 128)

- We need to preprocess the image and bring down the width to height ratio to 1:2. The image size should preferably be 64 x 128.
- This is because we will be dividing the image into 8*8 and 16*16 patches to extract the features.



Step 2: Calculating Gradients (direction x and y)

The next step is to calculate the gradient for every pixel in the image. **Gradients are the small changes in the x and y directions.**

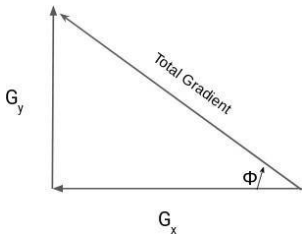


121	10	78	96	125
48	152	68	125	111
145	78	85	89	65
154	214	56	200	66
214	87	45	102	45

- Get the pixel values for this patch.*(the matrix shown here is only used as an example).*
- For the pixel value 85. To determine the gradient in the x-direction, subtract the value on the left from the pixel value on the right. To calculate the gradient in the y-direction, subtract the pixel value below from the pixel value above the selected pixel.
- Hence the resultant gradients in the x and y direction for this pixel are:
    - Change in X direction(G_x) = 89 – 78 = 11
    - Change in Y direction(G_y) = 68 – 56 = 8
- This process will give us two new matrices – one storing gradients in the x-direction and the other storing gradients in the y direction.
- The next step would be to find the magnitude and orientation using these values.

Step 3: Calculate the Magnitude and Orientation

Using the gradients we determine the magnitude and direction for each pixel value. For this step, we will use the Pythagoras theorem .



The gradients are basically the base and perpendicular here. So, for example, we had Gx and Gy as 11 and 8. Total gradient magnitude:`Total Gradient Magnitude` = $\sqrt{[(G_x)^2+(G_y)^2]}$ `Total Gradient Magnitude` = $\sqrt{[(11)^2+(8)^2]}$ = 13.6
Next, calculate the orientation (or direction) for

the same pixel. tan(Φ) = Gy / Gx

Hence, the value of the angle would be: Φ = atan(Gy / Gx)

The orientation comes out to be 36 when we plug in the values. This way for every pixel value, we have the total gradient (magnitude) and the orientation (direction). We need to generate the histogram using these gradients and orientations.

Different Methods to Create Histograms using Gradients and Orientation

We take the angle or orientation on the x-axis and the frequency on the y-axis.

Method 1:We will take each pixel value, find the orientation of the pixel and update the frequency table.For Ex: the process for the highlighted pixel (85). Since the orientation for this pixel is 36, we will add a number against angle value 36, denoting the frequency:

121	10	78	96	125
48	152	68	125	111
145	78	85	89	65
154	214	56	200	66
214	87	45	102	45

Frequency						1									
Angle	1	2	3	4 ...	35	36	37	38	39....	175	176	177	178	179	180

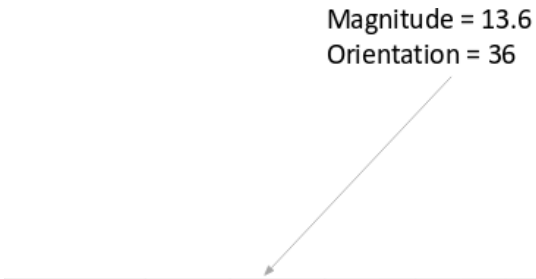
This frequency table can be used to generate a histogram with angle values on the x-axis and the frequency on the y-axis.

Method 2:Here we have a bin size of 20. So, the number of buckets we would get here is 9. For each pixel, store orientation into the frequency of the orientation values in the form of a 9 x 1 matrix. Plotting this would give us the histogram:

121	10	78	96	125
48	152	68	125	111
145	78	85	89	65
154	214	56	200	66
214	87	45	102	45

Magnitude		1							
Bin	0	20	40	60	80	100	120	140	160

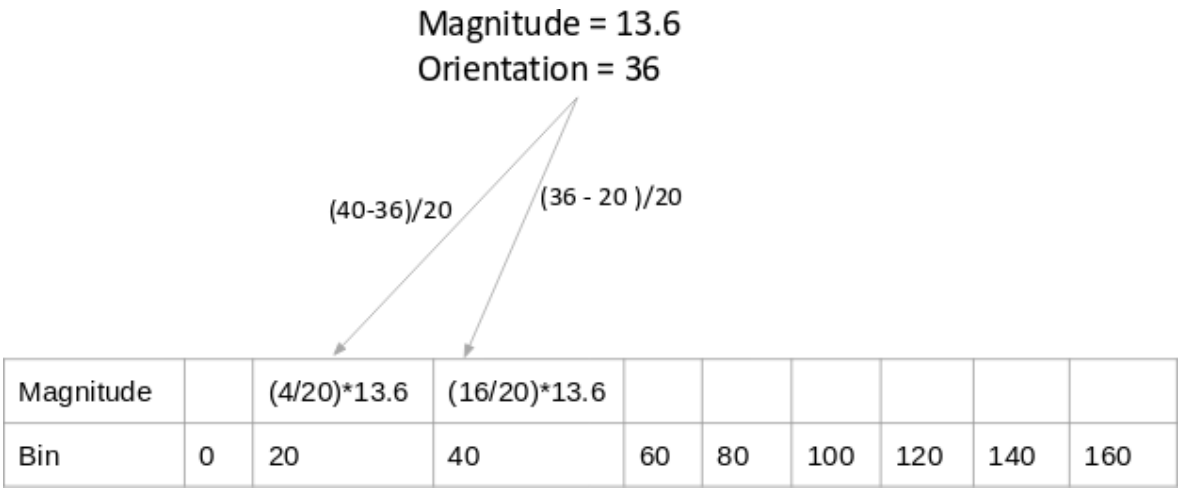
Method 3:Here is another way in which we can generate the histogram – instead of using the frequency, we can use the gradient magnitude to fill the values in the matrix**.**



Magnitude		13.6							
Bin	0	20	40	60	80	100	120	140	160

Here we are using the orientation value of 30, and updating the bin 20 only. Additionally, we should give some weight to the other bin as well.

Method 4:Let's make a small modification to the above method. Here, we will add the contribution of a pixel's gradient to the bins on either side of the pixel gradient. Remember, the higher contribution should be to the bin value which is closer to the orientation.

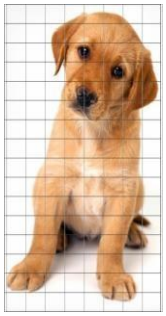


Magnitude		(4/20)*13.6	(16/20)*13.6						
Bin	0	20	40	60	80	100	120	140	160

Step 4: Calculate Histogram of Gradients in 8×8 cells (9×1)

The histograms created in the HOG feature descriptor are not generated for the whole image. Instead, the image is divided into 8×8 cells, and the histogram of oriented gradients is computed for each cell. If we divide the image into 8×8 cells and generate the histograms, we will get a 9 x 1 matrix for each cell. This matrix is generated using method 4 that

Once we have generated the HOG for the 8×8 patches in the image, the next step is to

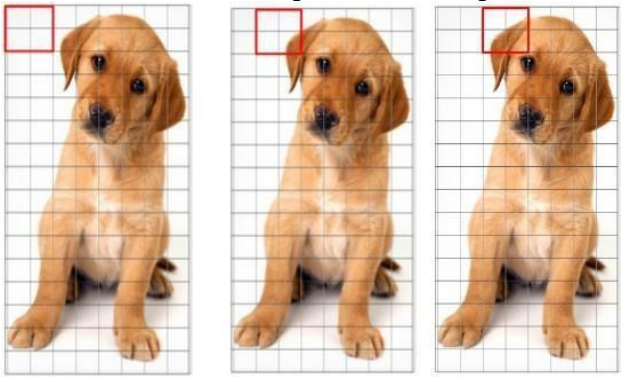


normalize the histogram.

Step 5: Normalize gradients in 16×16 cell (36×1)

The gradients of the image are sensitive to the overall lighting. This means that for a particular picture, some portion of the image would be very bright as compared to the other portions.

But we can reduce this lighting variation by normalizing the gradients by taking 16×16 blocks. Here is an example that can explain how 16×16 blocks are created:



Here, we will be combining four 8×8 cells to create a 16×16 block. And we already know that each 8×8 cell has a 9×1 matrix for a histogram. So, we would have four 9×1 matrices or a single 36×1 matrix. To normalize this matrix, Mathematically, for a given vector V = [a1, a2, a3, ….a36]

The root of the sum of squares: k = √(a1)2+

(a2)2+ (a3)2+ ⋯. (a36)2 And divide all the

values in the vector V with this value k:

$$\text{Normalised Vector} = \left(\frac{a_1}{k}, \frac{a_2}{k}, \frac{a_3}{k}, \ldots. \frac{a_{36}}{k} \right)$$

The resultant would be a normalized vector of size 36×1.

Step 6: Features for the complete image

Now, we will combine all (16 X16 blocks)these to get the features for the final image.

We would have 105 (7×15) blocks of 16×16. Each of these 105 blocks has a vector of 36×1 as features. Hence, the total features for the image would be 105 x 36×1 = 3780 features.